

Definition

A context-free grammar is a quadruple

$$G = (V, T, P, S)$$

where

V is a finite set of variables.

T is a finite set of terminals.

P is a finite set of productions of the form

$A \rightarrow \alpha$, where A is a variable and $\alpha \in (V \cup T)^*$

S is a designated variable called the start symbol.

Example :

An example grammar is

$$G = (V, T, P, S)$$

where

$$V = \{ S, A \} \quad T = \{ 0, 1 \}$$

and the productions,

P, are:

$$S \rightarrow 0A \mid 0$$

$$A \rightarrow 10A$$

This grammar corresponds to the regular expression $0(10)^*$ which in turn corresponds to the deterministic finite automata.

Regular Grammar

A regular grammar is defined as

$$G = (V, T, P, S)$$

where

V is a set of symbols called variables, $\{v_1, v_2, \dots, v_n\}$.

T is a set of symbols called terminal, $\{t_1, t_2, \dots, t_m\}$.

P is a set of production rules.

S is the starting or goal variable from V .

The productions P are of the form:

$$A \rightarrow w$$

$$A \rightarrow wB \quad \text{Where } A \text{ and } B \text{ are}$$

variables and w is a string made from any combination terminals, may be empty string. Any regular grammar can be converted to an equivalent DFA, NFA, regular language or regular expression.

CFG – Terminology

→ $L(G)$ is the language of G (the language generated by G) which is a set of sentences.

→ A sentence of $L(G)$ is a string of terminal symbols of G .

→ If S is the start symbol of G then w is a sentence of $L(G)$ iff $S \rightarrow w$ where w is a string of terminals of G .

→ If G is a context-free grammar, $L(G)$ is a context-free language.

→ Two grammars are equivalent if they produce the same language.

→ In, $S \rightarrow \alpha$

- If α contains non-terminals, it is called as a sentential form of G .

- If α does not contain non-terminals, it is called as a sentence of G .

Example 1:

$L = \{ a^n b^n \mid n \text{ is a positive integer} \}$
is a context-free language.

For the following context-free grammar
 $G_1 = (V, \Sigma, P, S)$ generates L
where

$$V = \{ S \},$$

$$\Sigma = \{ a, b \}$$

and $P = \{ S \rightarrow aSb, S \rightarrow ab \}.$

Example 2:

Let L be the set of algebraic expressions involving identifiers x and y , operations $+$ and $*$ and left and right parentheses.

Then,
 L is a context-free language.

For the following context-free grammar $G = (V, \Sigma, P, S)$ generates L where,

$$V = \{ S \},$$

$$\Sigma = \{x, y, (,), +, *\}$$

and $P = \{$

$$S \rightarrow (S+S),$$

$$S \rightarrow S*S,$$

$$S \rightarrow x,$$

$$S \rightarrow y$$

$$\}.$$

Derivations

Let a Grammar G is,

$\Rightarrow E \rightarrow E+E, E \rightarrow -(E) , E \rightarrow id$. Then,
 $E+E$ derives from E .

We can replace E by $E+E$ to able
 to do this, we have to have a
 production rule $E \rightarrow E+E$ in our
 grammar.

$\Rightarrow E \rightarrow E+E \rightarrow id+E \rightarrow id+id$

A sequence of replacements of non-
 terminal symbols is called a
 derivation of $id+id$ from E .

Example:

$E \rightarrow -E \rightarrow -(E) \rightarrow -(E+E) \rightarrow -(id+E) \rightarrow -(id+id)$

OR

$E \rightarrow -E \rightarrow -(E) \rightarrow -(E+E) \rightarrow -(E+id) \rightarrow -(id+id)$

At each derivation step, we can choose any of the non-terminal in the sentential form of G for the replacement.

If we always choose the left-most non-terminal in each derivation step, this derivation is called as **left-most derivation**.

If we always choose the right-most non-terminal in each derivation step, this derivation is called as **right-most derivation**.

Left-Most Derivation

$$E \rightarrow -E \rightarrow -(E) \rightarrow -(E+E) \rightarrow -(id+E) \rightarrow -(id+id)$$

Right-Most Derivation

$$E \rightarrow -E \rightarrow -(E) \rightarrow -(E+E) \rightarrow -(E+id) \rightarrow -(id+id)$$

Reduction

Process of reducing input string to starting Non-terminal using production rules.

$-(id+id) \rightarrow -(E+id) \rightarrow -(E+E) \rightarrow -(E) \rightarrow E$

Derivation Trees

A derivation tree of a Grammar G is constructed with,

1. each tree vertex is a variable or terminal or epsilon.
2. the root vertex is S .
3. interior vertices are from V , leaf vertices are from T or epsilon.
4. an interior vertex A has children, in order, left to right, $X_1, X_2, \dots, X_k$ when there is a production in P of the form $A \rightarrow X_1 X_2 \dots X_k$.
5. A leaf can be epsilon only when there is a production $A \rightarrow \epsilon$ and the leaf's parent can have only this child.

For any valid derivation tree, reading the leafs from left to right gives one string in the language defined by the grammar.

Ambiguous Grammars

→ the leftmost (rightmost) derivation tree is not unique for every string in the language defined by the grammar.

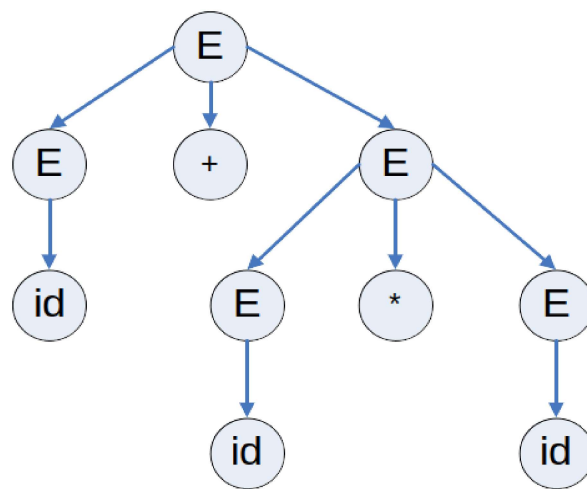
→ the leftmost and rightmost derivations are usually distinct but might be the same.

Example: Consider a grammar G,

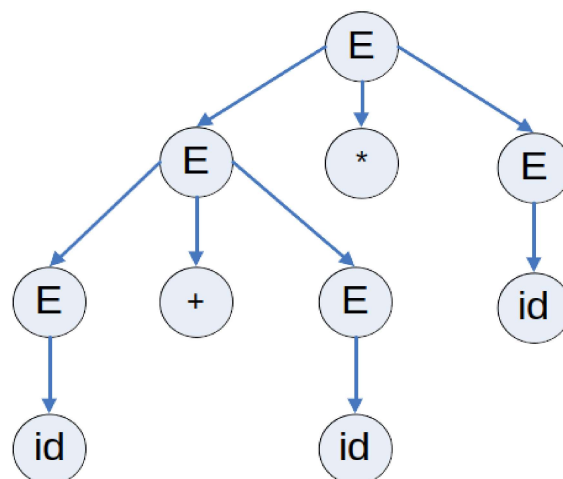
$$E \rightarrow E+E, E \rightarrow E * E, E \rightarrow id$$

We can construct derivation tree for $id+id*id$.

$$1. E \rightarrow E+E \rightarrow id+E \rightarrow id+E * E \rightarrow id+id * E \rightarrow id+id * id$$



$$2. E \rightarrow E * E \rightarrow E + E * E \rightarrow id + E * E \rightarrow id + id * E \rightarrow id + id * id$$



CFG simplification algorithm

The goal here is to take an arbitrary Context Free Grammar $G = (V, T, P, S)$ and perform transformations on the grammar that preserve the language generated by the grammar but reach a specific format for the productions. Overview of the steps:

Step 1a) Eliminate useless variables that can not become terminals

Step 1b) Eliminate useless variables that can not be reached

Step 2) Eliminate epsilon productions

Step 3) Eliminate unit productions

Step 4) Make productions Chomsky Normal Form

Step 5) Make productions Greibach Normal Form

The CYK parser uses Chomsky Normal Form as an input and the CFG to NPDA uses Greibach Normal Form as an input.

Details: one step at a time

1.

a) Eliminate **useless variables** that cannot become terminals

Basically: Build the set NEWV from productions of the form $V \rightarrow w$ where V is a variable and w is one or more terminals.

Insert V into the set NEWV. Then iterate over the productions, now accepting any variable in w as a terminal if it is in NEWV. Thus NEWV is all the variables that can be reduced to all terminals.

Now, all productions containing a variable not in NEWV can be thrown away. Thus T is unchanged, S is unchanged, $V = \text{NEWV}$ and P may become the same or smaller.

The new grammar $G = (V, T, P, S)$ represents the same language.

b) Eliminate **useless variables** that can not be reached from S .

Set $V' = S$, $T' = \emptyset$, mark all production as unused.

Iterate repeatedly through all productions until no change in V' or T' . For any production $A \rightarrow w$, with A in V' insert the terminals from w into the set T' and insert the variables from w into the set V' and mark the production as used.

Now, delete all productions from P that are marked unused. $V=V'$, $T=T'$, S is unchanged.

The new grammar $G = (V, T, P, S)$ represents the same language.

2) Eliminate **epsilon productions**.

This is complex. If the language of the grammar contains the null string, epsilon, then in principle remove epsilon from the grammar, eliminate epsilon productions, then put $S \rightarrow \epsilon$ back into the grammar later.

The new grammar $G = (V, T, P, S)$ represents the same language except the new language does not contain epsilon.

3) Eliminate **unit productions**.

Iterate through productions finding $A \rightarrow B$ type **unit productions**. Delete this production from P . Make a copy of all productions $B \rightarrow \gamma$, replacing B with A . Be careful of $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$ type cases, there needs to be copies of $B \rightarrow \gamma$, $C \rightarrow \gamma$, $D \rightarrow \gamma$ for A . Delete duplicate

productions. (Sort and remove adjacent duplicate)

The new grammar $G = (V, T, P, S)$ represents the same language.

Example : Removal of ϵ -productions.

$$\begin{aligned} S &\rightarrow ABCD \\ A &\rightarrow aA \mid bB \mid \epsilon \\ B &\rightarrow bB \mid cC \mid \epsilon \\ C &\rightarrow cC \mid cD \mid \epsilon \\ D &\rightarrow dD \mid \epsilon \end{aligned}$$

To remove ϵ -productions, variables A, B, C, D on the RHS can be eliminated 1, 2, 3 or all in turn. Thus,

$$\begin{aligned} S &\rightarrow ABCD \mid BCD \mid ACD \mid ABC \mid ABC \mid \\ &\quad CD \mid BD \mid AB \mid AC \mid AD \mid BC \mid \\ &\quad A \mid B \mid C \mid D \\ A &\rightarrow aA \mid bB \mid a \mid b \\ B &\rightarrow bB \mid cC \mid b \mid c \\ C &\rightarrow cC \mid cD \mid c \\ D &\rightarrow dD \mid d \end{aligned}$$

has all the ϵ productions removed and is equivalent to the original grammar.

Example : Removal of ϵ -productions.

$$S \rightarrow AaBb$$

$$A \rightarrow abB \mid \varepsilon$$

$$B \rightarrow ABb \mid A \mid \varepsilon$$

To remove ε productions of $A \rightarrow \varepsilon$ and $B \rightarrow \varepsilon$ wherever A and B occurs, additional productions without A or B or both are included.

Thus,

$$S \rightarrow AaBb \mid aBb \mid Aab \mid ab$$

$$A \rightarrow abB \mid ab$$

$$B \rightarrow ABb \mid Ab \mid Bb \mid b \mid abB \mid ab$$

has all the ε productions removed and is equivalent to the original grammar.

Chomsky Normal Form

Chomsky Normal Form (CNF) of grammar is a CFG.

$G = (V, T, P, S)$ such that all productions are either of the two types, $A \rightarrow BC$ or $A \rightarrow a$ Where A, B and C are variables and $a \in \Sigma$.

In overall, simplification of grammar is to convert it to Chomsky Normal Form. For this, the productions can be one of two formats $A \rightarrow a$ or $A \rightarrow BC$. The right side of the production is either exactly one terminal symbol or exactly two variables. The grammars must have the simplification steps:

- (a) No Useless variables,
- (b) No Null-able variables and
- (c) No Unit productions.

Example :

$$S \rightarrow AaBb \mid aBb \mid Aab \mid ab$$

$$A \rightarrow abB \mid ab$$

$$B \rightarrow ABb \mid Ab \mid Bb \mid b \mid abB \mid ab$$

In the next step, all the terminal symbols on the RHS are replaced by equivalent non terminals, such as a by X_a and b by X_b .

We get,

$$S \rightarrow AX_aBX_b \mid X_aBX_b \mid AX_ab \mid X_aX_b$$

$$A \rightarrow X_aX_bB \mid X_aX_b$$

$$B \rightarrow ABX_b \mid AX_b \mid BX_b \mid b \mid X_aX_bB \mid X_aX_b$$

$$X_a \rightarrow a$$

$$X_b \rightarrow b$$

Now, The next step will be to combine all NT's more than two on the RHS such as,

$$S \rightarrow \gamma_a\gamma_b \mid X_a\gamma_b \mid \gamma_ab \mid X_aX_b$$

$$A \rightarrow \gamma_{ab}B \mid X_aX_b$$

$$B \rightarrow ABX_b \mid AX_b \mid BX_b \mid b \mid \gamma_{ab}B \mid X_aX_b$$

$X_a \rightarrow a$ and $X_b \rightarrow b$.

With $\gamma_a \rightarrow AX_a$, $\gamma_b \rightarrow BX_b$ and $\gamma_{ab} \rightarrow X_aX_b$. Now the grammar is in the form either $S \rightarrow AB$ or $S \rightarrow a$ as required by CNF.

Greibach Normal Form

A CFG is in Greibach Normal Form if all productions are of the type

$$A \rightarrow aV$$

with $a \in T$ and $V \in V^*$. (Where 'A' is a variable, 'a' is exactly one terminal and 'V' is a string of none or more variables.) Conversion to this type is much easier than to the Chomsky form.

Example :

Consider the same grammar,

$$S \rightarrow AaBb$$

$$A \rightarrow abB \mid \epsilon$$

$$B \rightarrow ABb \mid A \mid \epsilon$$

Adding production $C \rightarrow a$ and $D \rightarrow b$ we get,

$$S \rightarrow ACBD$$

$$A \rightarrow aDB \mid \epsilon$$

$$B \rightarrow ABD \mid A \mid \epsilon$$

After removing ϵ productions we have,

$$S \rightarrow ACBD \mid CBD \mid ACD \mid CD$$

$$A \rightarrow aDB \mid aD$$

$$B \rightarrow ABD \mid AD \mid BD \mid b$$
$$C \rightarrow a$$
$$D \rightarrow b$$

Which is now in Greibach Normal Form.

CW:

1. Convert the grammar

$$S \rightarrow aSaA \mid A$$
$$A \rightarrow abA \mid b \quad \text{into CNF and GNF.}$$

2. Let G be the grammar

$$S \rightarrow aB \mid bA$$
$$A \rightarrow a \mid aS \mid bAA$$
$$B \rightarrow b \mid bS \mid aBB \quad \text{for the string } aaabbabbba.$$

Find the leftmost derivation and the right most derivation.